Interview Cassandra 2

What are the advantages of user-defined data types over built-in types?

User-defined data types (UDTs) in Cassandra offer several advantages over built-in types:

1. **Customization**: UDTs allow you to create a data type that fits your specific needs. For example, you can define an address type that includes fields for street, city, and zip code, which is not possible with built-in types.
2. **Grouping Related Data**: UDTs enable you to group multiple related fields into a single column. This is useful for representing complex data structures, like an address or a product with multiple attributes, making your data model cleaner and more organized.
3. **Flexibility**: You can modify UDTs by adding or removing fields as your application requirements change, providing greater flexibility compared to fixed built-in types.
4. **Improved Readability**: Using UDTs can make your data model more intuitive and easier to understand, as they can represent real-world concepts more closely than built-in types.
5. **Support for Collections**: UDTs can include built-in types and even other UDTs, allowing for complex nested structures that can represent more intricate relationships in your data.

What are the limitations of built-in data types in Cassandra?

Built-in data types in Cassandra have some limitations, including:

1. **Fixed Structure**: Built-in data types are predefined and cannot be modified. This means you cannot customize them to fit specific application needs or complex data structures.
2. **Limited Complexity**: They are suitable for simple data types like integers, strings, and booleans, but they cannot represent more complex data structures that require multiple related fields.
3. **No Grouping**: Built-in types cannot group related data together. For example, if you want to store an address with street, city, and zip code, you would need to use separate columns for each piece of information instead of a single, cohesive structure.
4. **Lack of Flexibility**: Once you choose a built-in data type for a column, you cannot change it without dropping the column and recreating it, which can be cumbersome and may lead to data loss.
5. **No Nested Structures**: Built-in types do not support nesting, meaning you cannot create a data type that contains other data types, limiting the ability to represent hierarchical data.

How can you use collection data types in a project?

You can use collection data types in a project to efficiently manage and store related data. Here are a few practical examples:

1. **User Profiles**:
   - **Lists**: Store a user's favorite movies in a list. This allows you to maintain the order of the movies and even allow duplicates (e.g., if a user loves a particular movie).
   - **Example**:

```
CREATE TABLE user_profiles (
    user_id UUID PRIMARY KEY,
```

```
    favorite_movies LIST<TEXT>
);
```

2. **Product Catalog**:
    - **Maps**: Use a map to store product attributes, where the key is the attribute name (like "color" or "size") and the value is the attribute value. This allows for flexible product descriptions.
    - **Example**:

```
CREATE TABLE products (
    product_id UUID PRIMARY KEY,
    attributes MAP<TEXT, TEXT>
);
```

3. **Event Management**:
    - **Sets**: Store unique participants in an event using a set. This ensures that each participant is listed only once, avoiding duplicates.
    - **Example**:

```
CREATE TABLE events (
    event_id UUID PRIMARY KEY,
    participants SET<TEXT>
);
```

Using collection data types helps you avoid complex joins and keeps related data together, making your data model more efficient and easier to manage.

What is a keyspace in Apache Cassandra?

A **keyspace** in Apache Cassandra is a fundamental concept that serves as a container for your data tables. Here are the key points about keyspaces:

- **Definition**: A keyspace is a namespace that defines how data is replicated across the nodes in a Cassandra cluster.
- **Tables**: You must create a keyspace before you can create any tables. Each keyspace can contain multiple tables, but each table belongs to only one keyspace.
- **Replication**: When creating a keyspace, you specify the **replication factor** (the number of copies of the data) and the **replication strategy** (how those copies are distributed across different nodes).
- **No Default Keyspace**: There is no default keyspace in Cassandra, so you need to define one explicitly.

In summary, a keyspace is essential for organizing your data and ensuring its availability and durability through replication.

What would happen if you used Lightweight Transactions frequently?

Using **Lightweight Transactions (LWT)** frequently in Cassandra can lead to some performance issues. Here's a simple breakdown:

- **Slower Performance**: LWTs are at least four times slower than regular INSERT and UPDATE operations. This is because they require Cassandra to read the data before performing the write, which adds extra steps to the process.
- **Increased Resource Usage**: Since LWTs involve more complex operations, they can consume more resources (like CPU and memory) on the nodes in your cluster. This can lead to higher latency and slower response times for your database operations.
- **Potential Bottlenecks**: If many transactions are trying to use LWT at the same time, it can create bottlenecks, slowing down the overall performance of your database.

In summary, while LWTs are useful for ensuring data integrity (like checking if a record exists before updating it), using them too often can negatively impact the performance of your Cassandra database. It's best to use them sparingly and only when necessary.

How could you optimize the use of Lightweight Transactions?

To optimize the use of **Lightweight Transactions (LWT)** in Cassandra, consider the following strategies:

1. **Limit Usage**:
    - Use LWTs only when absolutely necessary, such as when you need to ensure data integrity (e.g., checking if a record exists before updating).
2. **Batch Operations**:
    - Combine multiple LWTs into a single transaction when possible. This reduces the number of times Cassandra has to perform the read-before-write operation.
3. **Use Regular Writes When Possible**:
    - If you don't need the guarantees provided by LWTs, opt for regular INSERT or UPDATE operations. This will improve performance significantly.
4. **Optimize Data Model**:
    - Design your data model to minimize the need for LWTs. For example, structure your data in a way that reduces contention and the need for checks before writes.
5. **Monitor Performance**:
    - Keep an eye on the performance metrics of your Cassandra cluster. If you notice slowdowns, evaluate the frequency and necessity of LWTs in your application.
6. **Use Appropriate Consistency Levels**:

- Choose the right consistency level for your operations. Sometimes, a lower consistency level can suffice, allowing you to avoid LWTs.

By implementing these strategies, you can effectively manage the use of Lightweight Transactions and maintain better performance in your Cassandra database.

What is the main purpose of Lightweight Transactions in Cassandra?

The main purpose of **Lightweight Transactions (LWT)** in Cassandra is to ensure **data consistency and integrity** during write operations. Here are the key points:

- **Conditional Writes**: LWTs allow you to perform operations that depend on certain conditions. For example, you can update a record only if it exists or insert a new record only if it does not already exist.
- **Avoiding Race Conditions**: They help prevent race conditions in scenarios where multiple clients might try to modify the same data simultaneously. LWTs ensure that only one operation can succeed based on the specified conditions.
- **Data Integrity**: By enforcing checks before writes, LWTs help maintain the accuracy and reliability of the data in the database.

In summary, Lightweight Transactions are used when you need to perform operations that require a guarantee about the state of the data before making changes, ensuring that your data remains consistent and valid.

What is a tombstone in Cassandra?

A **tombstone** in Apache Cassandra is a special marker used to indicate that a piece of data has been deleted. Here's how it works in simple terms:

- When you delete data in Cassandra, instead of removing it completely, the system writes a tombstone. This tombstone acts like a note saying, "This data is no longer valid."
- The tombstone includes information about when the deletion occurred, which helps the system manage data consistency across different nodes in the cluster.

For example, if you delete a user's information, a tombstone will be created to show that this user's data has been deleted. Even though the data is not visible in queries anymore, it still exists on disk for a certain period. This is important because it allows the system to handle any inconsistencies that might arise from the distributed nature of Cassandra.

Tombstones are eventually removed from the system after a set period, known as **gc_grace_seconds** (default is 10 days). This helps keep the database clean and efficient.

How does a tombstone affect data consistency in Cassandra?

A tombstone affects data consistency in Cassandra in several ways:

1. **Indicates Deletion**: When a tombstone is created, it signals to the system that a specific piece of data has been deleted. This helps maintain consistency across different nodes in the cluster, as all nodes will eventually recognize that the data is no longer valid.
2. **Handling Inconsistencies**: In a distributed system like Cassandra, data can be stored on multiple nodes. If a read request comes in, the coordinator node checks the responses from different nodes. If one node has the original data and another has a tombstone, the tombstone (which indicates a more recent operation) will take precedence. This ensures that the most up-to-date information is returned to the user.
3. **Temporary Data Presence**: Even after a tombstone is created, the actual data remains on disk for a period of time. This allows the system to resolve any

potential conflicts that may arise from different nodes having different versions of the data. It ensures that if a read request is made shortly after a deletion, the system can still provide consistent results.

4. **Performance Considerations**: While tombstones help maintain consistency, having too many tombstones can negatively impact performance. They take up space and can slow down read operations, as the system has to check for tombstones when processing queries. Therefore, it's important to manage tombstone creation and deletion effectively.

In summary, tombstones play a crucial role in ensuring data consistency in Cassandra by marking deleted data, helping resolve conflicts, and maintaining the integrity of the database.

1. What characterizes keyspaces in Apache Cassandra?

- ○ A keyspace can contain a finite number of tables.
- ○ A table can belong to any number of keyspaces.
- ○ A keyspace must be defined after creating tables, as there is no default keyspace.
- ◉ Replication is specified at the keyspace level.

Correct! Replication is specified at the keyspace level.

✅ **Replication is specified at the keyspace level.**

Explanation:

In **Apache Cassandra**, a **keyspace** is a top-level container that holds tables (similar to a database in relational systems). Here's what characterizes keyspaces:

1. **Replication is specified at the keyspace level** – When creating a keyspace, you define the **replication strategy** (SimpleStrategy or NetworkTopologyStrategy) and the **replication factor**, which determines how many copies of the data are stored across the cluster.
2. **A keyspace must be defined before creating tables** – Unlike some databases that have a default schema, Cassandra requires you to explicitly create a keyspace before defining tables.
3. **A table belongs to exactly one keyspace** – Tables are scoped to a single keyspace, not multiple.
4. **A keyspace can contain many tables** – There is no strict "finite" limit (other than practical system constraints).

Incorrect Options:

- ❌ *"A keyspace must be defined after creating tables"* → **False**, tables cannot exist without a keyspace.
- ❌ *"A table can belong to any number of keyspaces"* → **False**, a table is tied to a single keyspace.
- ❌ *"A keyspace can contain a finite number of tables"* → While technically true (due to system limits), this is not a defining characteristic—Cassandra allows a keyspace to hold many tables.

Thus, the **most accurate and defining characteristic** is that **replication is configured at the keyspace level.** ✅

2. What is one way to get Apache Cassandra to locate and read data? 1 /

- ○ Use UPDATE data with Time-To-Live
- ○ Use UPDATE data with the full primary key specified
- ◉ Use Lightweight Transactions

  Correct! You can instruct Cassandra to look for the data, read it, and only then perform a given operation by using Lightweight Transactions, but Lightweight Transactions are slower than the normal INSERT/UPDATE in Cassandra.

- ○ Use INSERT data with Time-To-Live

1. Which CQL data type is typically used to store an image or audio?

- ○ Bigint
- ○ Maps
- ◉ Blobs
- ○ Sets

Correct
Correct! Blobs are typically used to store images, audio, or other multimedia objects.

2. Which command would you use to add new columns to a table schema?

- ○ TRUNCATE TABLE
- ○ CREATE TABLE
- ◉ ALTER TABLE
- ○ DROP TABLE

Correct
Correct! ALTER TABLE can add new columns to a table schema.

✅ Blobs

- **Explanation:** The `BLOB` (Binary Large Object) data type is used to store binary data like images, audio, or other files.

❌ Incorrect options:

- `Bigint` → Stores large integers, not binary data.
- `Maps` → Stores key-value pairs, not raw binary data.
- `Sets` → Stores a collection of unique values, not binary data.

Question 2

Which command would you use to add new columns to a table schema?

✅ **ALTER TABLE**

- **Explanation:** `ALTER TABLE` modifies an existing table's schema (e.g., adding/dropping columns).

❌ Incorrect options:

- `TRUNCATE TABLE` → Deletes all data but keeps the schema.
- `CREATE TABLE` → Makes a new table, doesn't modify existing ones.
- `DROP TABLE` → Deletes the table entirely.

3. What does introducing an IF in INSERT and UPDATE statements make possible?

- ○ Attaches a timestamp to a specific write operation
- ○ Creates a new file called SSTable
- ◉ Instructs Cassandra to look for the data, read it, and only then perform a given operation
- ○ Inserts or updates data with a specific Time-To-Live

Correct
Correct! This is possible through lightweight transactions, which are supported syntax-wise by introducing IF in INSERT and UPDATE statements.

4. Select the answer that describes a difference between CQL and SQL.

- ○ Syntax that allows the creation of inserts, deletes, and select queries
- ◉ Support of JOIN statements
- ○ Intuitive and simple syntax
- ○ Syntax that allows the creation of tables and keyspaces

Correct
Correct! CQL does not support JOIN statements. You can store the data already joined.

Question 3

What does introducing an IF in INSERT and UPDATE statements make possible?

✅ **Instructs Cassandra to look for the data, read it, and only then perform a given operation**

- **Explanation:** `IF` enables **Lightweight Transactions (LWTs)**, which check conditions (e.g., `IF EXISTS`) before executing the operation.

❌ Incorrect options:

- Attaching timestamps → Done automatically, not via `IF`.
- Creating SSTables → A storage engine process, unrelated to `IF`.
- TTL (Time-To-Live) → Set via `USING TTL`, not `IF`.

Question 4

Select the answer that describes a difference between CQL and SQL.

✅ **Support of JOIN statements**

- **Explanation:** Unlike SQL, **CQL does not support JOINs** due to Cassandra's distributed design (denormalization is preferred).

❌ Incorrect options:

- Syntax for inserts/deletes → Both CQL and SQL support these.
- Intuitive syntax → Subjective; both have simple syntax.
- Creating tables/keyspaces → Both allow schema creation.

5. What does DROP KEYSPACE remove?

- (●) All keyspace tables and their data
- ( ) All keyspace table data
- ( ) All data and primary keys
- ( ) All data and schema

Correct
Correct! Using DROP KEYSPACE will lead to the removal of all the keyspace tables and the data they contain.

Question 5

What does DROP KEYSPACE remove?

✅ **All keyspace tables and their data**

- **Explanation:** `DROP KEYSPACE` deletes the **entire keyspace**, including all tables and their data permanently.

❌ Incorrect options:

- Only table data → `TRUNCATE` removes data but keeps schema.
- Primary keys → Primary keys are part of the schema, which is dropped entirely.
- All data and schema → Close, but the phrasing implies partial removal (DROP removes *everything*).